Agentic Progressive Optimization for Hardware–Software Co-Design of AI Infrastructure

Research manuscript

October 10, 2026

Abstract

A software optimization can become worthwhile only after the hardware changes. Larger tiles need sufficient registers, fusion changes storage lifetimes, and reuse depends on the memory system's ability to serve concurrent work. Designing AI infrastructure therefore requires choosing algorithms, programs, interfaces, and hardware together. It also requires a representation that makes useful combinations available to the optimizer. We develop Agentic Progressive Co-Design around an evolving mixed-integer program (MIP). Agents enrich coarse computation graphs with tensor dependencies and implementation alternatives, introduce software–hardware boundary choices, and restructure the resulting formulation to make coupled search tractable. One assignment specifies both a program and its matching hardware configuration; execution feedback guides which alternatives to retain and which model assumptions to revise. Streaming attention and programmable transfer mechanisms illustrate how the formulation connects algorithmic choices to hardware cost and capability. Preliminary experiments achieve a 7.02× aggregate speedup on a programmable architecture benchmark and 2.93× prefill and 3.58× decode speedups on the open-source Ventus GPU architecture. Controlled comparisons show gains from optimizing either software or hardware, with larger gains when both can change. Hardware changes make previously unfavorable tiling and attention mappings useful, while model revisions expose additional beneficial combinations. These results motivate a design process that revisits earlier choices as its representation becomes more expressive, while allocating search effort to the interactions that determine implementation quality.

1 Introduction

A hardware resource has value only through the programs that use it. Larger tiles can reduce repeated weight transfers, yet increase register demand enough to reduce occupancy. More local storage can enable reuse, yet consume area that could support additional execution units or outstanding memory requests. Fusing two operators can remove intermediate transfers, yet change the instruction mix and the resources that limit progress. These choices jointly determine the efficiency of AI infrastructure.

The interactions are visible even within a small design family. In a Ventus study, increasing a matrix-multiplication tile from 32×64 to 64×64 reduced repeated weight traffic, but the new kernel required 173 vector registers. It performed poorly on the preceding register configuration and became useful after the hardware allocation changed. An attention mapping rejected on the default machine subsequently improved execution on another configuration (Section 6.3). Such reversals expose a limitation of committing to software or hardware choices before their interactions are understood.

Layered design makes a large problem manageable. A computation graph describes operators, a compiler constructs programs, an instruction interface specifies executable operations, and a hardware implementation supplies resources. Each layer hides details that other layers need not routinely consider. However, a hidden detail can become a consequential design variable. A coarse attention graph

specifies matrix multiplication and normalization without exposing whether the score matrix is materialized, which tiles remain live, or how partial normalization state is updated. Once these choices become explicit, the available algorithms and the hardware needed to run them can change together.

This leads to two connected difficulties. *Representation* determines which designs can be considered. An optimizer cannot select an online reduction, a different communication pattern, or a specialized data path that its representation does not admit. *Search* determines which of the admitted designs can be found within the available effort. Expressing all relevant tiling, storage, scheduling, instruction, and hardware choices at once can produce a mixed-integer program (MIP) with an impractical number of variables and weak relaxations. Improving only one side leaves the other as a bottleneck.

We organize this process as Agentic Progressive Co-Design. Agents construct and revise the mathematical design problem, while optimization solvers search its explicit choices. The overall domain includes algorithmic organization, executable programs, the software–hardware boundary, and hardware structure. The active formulation exposes a selected part of that domain. Agents can introduce a new algorithmic family, expand a resource abstraction, derive a stronger encoding, or reopen a group of coupled decisions. They retain the assumptions behind reductions so that an unfavorable result on one machine does not eliminate a useful implementation elsewhere.

Three foundations suggest how to organize this process. CoSA shows how regular computation and architectural constraints support MIP-based scheduling [1]. Axon demonstrates semantics-driven generation of tensor-program alternatives and instruction implementations [2]. LLM4MIP demonstrates a complementary role for language models in analyzing difficult optimization instances and developing primal and dual strategies[1]. Our framework connects these directions around a changing software–hardware design domain: the program family, hardware capabilities, and optimization representation can all be revised.

The software–hardware boundary is an explicit part of that domain. A transfer pipeline may be orchestrated by software, exposed through programmable asynchronous instructions, or managed by a specialized controller. These alternatives change instruction overhead, queue requirements, control area, and the range of workloads supported. Choosing among them requires the same joint treatment as choosing tile dimensions and memory capacity. The objective is a matched implementation: an algorithm and program organization, an execution interface, and a hardware structure that together satisfy the workload and deployment constraints.

The paper develops this design process in three stages. Section 3 defines the relationship between the bounded implementation domain and the designs represented by an active MIP. This distinction makes representation expansion, hardware-dependent dominance, and branch retention precise. Sections 4 and 5 turn the formulation into a search procedure: tensor dependencies connect algorithmic alternatives to resource constraints, boundary variables price specialization, and solver and execution feedback guide subsequent model changes. Section 6 then examines these choices through a programmable architecture benchmark and Ventus experiments, using performance gains, preference reversals, storage tradeoffs, and model revisions to explain the value of joint design.

The empirical studies exercise executable software alternatives, parameterized hardware models, and MIP-guided selection. They establish the interactions that a progressive formulation must capture. Autonomous construction of the full hierarchy and generation of new instruction interfaces extend beyond the evaluated system; the formulation identifies how to incorporate them. Functional acceptance remains a prerequisite for comparing candidate designs.

[1]LLM4MIP released workflows: https://llm4mip.github.io/.

2 Related Work

Mathematical optimization of program decisions. CoSA encodes spatial and temporal scheduling choices for DNN accelerators as a MIP, using algorithmic and architectural constraints [1]. It provides a concrete basis for turning loop structure and resource limits into solver decisions. Checkmate uses mixed-integer optimization for tensor rematerialization [3], while COSMA jointly optimizes operator scheduling, memory allocation, and tensor replacement for a specified scratchpad capacity [4]. These systems demonstrate that decisions often handled by compiler heuristics can be represented mathematically. Our focus is how the program families, hardware resources, and interfaces underlying such decisions can evolve together, and how the formulation should change when those alternatives become relevant.

Generating and representing optimized tensor programs. MLIR supports multiple abstraction levels and domain-specific intermediate representations [5]. TensorIR makes tensor computation primitives explicit within a compiler representation [6]. Ansor searches generated tensor-program schedules using measurement and a learned cost model [7]. Mirage represents programs at kernel, thread-block, and thread levels to search combinations of algebraic and scheduling transformations [8]. These approaches establish that the representation is a central part of optimization: it determines the transformations that can be combined and the dependencies that remain visible.

Axon is particularly relevant to software-space construction. It generates tensor-graph alternatives, synthesizes target instruction sequences from semantics, and explores tiling and fusion for tile-based accelerators [2]. Its persistent representation retains alternatives through successive stages. These capabilities suggest a source of implementation families for a global co-design formulation. Our design additionally exposes hardware resources and the implementation location of selected mechanisms as decisions. The generated family therefore has both software requirements and hardware realizations, rather than being ranked only on one target machine.

Joint architecture and mapping search. DiGamma searches hardware and software mappings with a domain-aware genetic algorithm [9]. DOSA uses differentiable performance models to explore mappings and hardware together [10]. These works are direct precedents for joint optimization; hardware–software co-design itself is an established problem. We investigate a different control layer: how agents introduce design mechanisms, revise mathematical representations, and select structured subproblems. A fair evaluation must compare against strong joint and alternating search, in addition to one-pass layer-wise optimization.

Language models and optimization formulations. OptiMUS constructs and solves mathematical optimization models from problem descriptions [11]. FormuEvo searches for solver-efficient formulations using language-model proposals and solver feedback [12]. LLM4MIP studies already formulated MIP instances and uses structural analysis to develop feasible solutions and stronger bounds. These directions motivate a formulation agent that reasons about dependent choices and auxiliary variables instead of directly predicting a large assignment. Co-design adds two responsibilities: discovering implementation choices that the current model does not express, and comparing the model's ranking with execution evidence.

Hardware generation and evaluation. Timeloop evaluates accelerator architectures and mappings [13]; Gemmini connects a parameterized accelerator generator with its software stack [14]; FireSim supports FPGA-accelerated system simulation [15]. Such infrastructure provides the realization and evaluation paths required after optimization. It also supplies information that a compact MIP may omit, including compiler resource allocation and shared-resource timing. Our framework uses this feedback to revise representation and cost assumptions. Its functional acceptance interface can employ existing checking tools, without making proof construction or proof reuse part of the optimization

contribution.

3 Co-Design as Progressive Design-Space Construction

3.1 Workloads, implementations, and a shared machine

Let $\mathcal{W}$ be a finite set of deployment scenarios. A scenario specifies the required computation, tensor dimensions, data types, permitted numerical error, and persistent state. Prefill and a bounded sequence of decoding steps are separate scenarios, even when they use the same model weights. The workload fixes the required outputs; it leaves implementation choices open.

A design is

$$d = (h, b, \{a_w, p_w, m_w, \sigma_w\}_{w \in \mathcal{W}}), \tag{1}$$

where h is the hardware structure, b its software-visible boundary, a_w an algorithmic organization, p_w a program, m_w its mapping, and σ_w its execution organization. Hardware and interface choices are shared across scenarios. Programs and schedules can differ. This separates specialization to one shape from building a machine that must support several shapes or execution modes.

The implementation domain $\mathcal{D}$ is bounded by a declared vocabulary: permitted arithmetic primitives, transformation families, storage structures, interface mechanisms, parameter ranges, and program-size limits. A design belongs to $\mathcal{D}_W \subseteq \mathcal{D}$ when its programs meet the workload contract and its hardware and execution satisfy the deployment constraints. A representative objective is

$$\min_{d \in \mathcal{D}_W} J(d), \qquad J(d) = \max_{w \in \mathcal{W}} \frac{L_w(d)}{L_w^{\mathrm{ref}}}, \quad A(h, b) \le A_{\max}, \tag{2}$$

with fixed positive reference latencies. Energy, throughput, or weighted latency can replace this objective. Retaining programmability can be expressed through required scenario coverage or supported capabilities. For a single fixed workload, specialization is a legitimate outcome unless flexibility has an explicit value or constraint.

The joint domain can be viewed as one large mixed-integer design problem once the structural vocabulary and numerical approximations are fixed. This view does not require materializing that formulation in full. The computational task is to expose and solve useful portions while retaining the links needed to combine their decisions.

3.2 Representations determine the reachable designs

An active formulation is

$$F_k = (R_k, X_k, C_k, \widehat{J}_k, D_k), \qquad \mathcal{F}_k = \{x \in X_k : C_k(x)\}, \tag{3}$$

where R_k is the representation, X_k the variable domain, C_k the constraints, $\widehat{J}_k$ the modeled objective, and D_k a decoder into implementations. Its realizable set is $\mathcal{R}_k = D_k(\mathcal{F}_k)$. A decoder retains the identities of the selected software and hardware objects; a feasible assignment must be lowerable under the corresponding implementation contracts.

A coarse representation can describe a broad mathematical computation while providing a narrow set of executable lowerings. For example, an attention node denotes many possible implementations, but a backend that always materializes its score matrix cannot generate a streaming implementation. Refinement exposes state and dependencies that make another lowering expressible. It need not change the required computation.

There are four distinct modifications of an active model:

- *Representation expansion* introduces additional realizable designs, such as an online reduction or a programmable transfer mechanism.
- *Equivalent reformulation* changes the encoding while preserving the decoded feasible designs and their objective values.
- *Restriction* removes a subset of designs through a justified dominance rule or a temporary search decision.
- *Cost or mechanism revision* changes the predicted behavior, constraints, or resource interactions associated with designs.

These operations need not produce nested feasible sets. Adding a new implementation family expands the design domain; modeling a previously omitted port conflict can remove assignments; changing coefficients can alter the ranking without changing feasibility. The system records these effects separately.

Proposition 1 (Expansion under a fixed evaluator)**.** *Suppose $\mathcal{R}_k \subseteq \mathcal{R}_{k+1} \subseteq \mathcal{D}_W$, both sets are nonempty, and the same objective J is used. Then* $\min_{d\in\mathcal{R}_{k+1}} J(d) \le \min_{d\in\mathcal{R}_k} J(d)$.

Proof. Every design admitted before expansion remains a candidate afterward, including any minimizer of the original finite design set. □

The proposition describes representational opportunity, not the quality achieved by a time-limited search. A richer representation can have a better optimum and still be harder to search. This distinction motivates retaining accepted incumbents and controlling where detail is introduced.

A useful decomposition separates the losses caused by representation, search, and prediction. Let $J^* = \min_{d\in\mathcal{D}_W} J(d)$ and $J_k^* = \min_{d\in\mathcal{R}_k} J(d)$. The representation loss is $J_k^* - J^*$. It is unaffected by additional solver effort within an unchanged $\mathcal{R}_k$.

Proposition 2 (Three sources of design loss)**.** *Let $\mathcal{R}_k \subseteq \mathcal{D}_W$ be finite and nonempty. Suppose $|\widehat{J}_k(d) - J(d)| \le \epsilon_k$ for every $d \in \mathcal{R}_k$, and the selected design $\widehat{d}_k$ satisfies $\widehat{J}_k(\widehat{d}_k) \le \min_{d\in\mathcal{R}_k} \widehat{J}_k(d) + \delta_k$. Then*

$$J(\widehat{d}_k) - J^* \le \underbrace{J_k^* - J^*}_{\textit{representation loss}} + \underbrace{\delta_k}_{\textit{optimization error}} + \underbrace{2\epsilon_k}_{\textit{prediction error}}. \tag{4}$$

Proof. For a true minimizer d_k^* over $\mathcal{R}_k$, $J(\widehat{d}_k) \le \widehat{J}_k(\widehat{d}_k) + \epsilon_k \le \widehat{J}_k(d_k^*) + \delta_k + \epsilon_k \le J_k^* + \delta_k + 2\epsilon_k$. Subtracting J^* gives the result. □

This standard perturbation argument clarifies the agent's different responsibilities. Introducing a missing algorithm can reduce representation loss; stronger formulations or better search can reduce optimization error; implementation feedback can improve prediction. The quantities need not be known globally to guide experiments. A formal use of the bound, however, requires a uniform error guarantee; residuals on a sampled set alone do not provide one. The Ventus model-error case illustrates why a small δ_k can coexist with a poor realized design.

3.3 Conditional dominance and alternative retention

Dominance is a property of a design in context. Let p and q be two program fragments with the same functional interface, and let Γ be a set of hardware configurations and surrounding programs. Removing p is safe only when every feasible completion using p in Γ has a feasible completion using q with no larger objective. Lower latency for one isolated kernel on one machine is insufficient: replacement can change storage lifetimes, interference, or other consumers' readiness.

Proposition 3 (Contextual replacement)**.** *For every feasible design d using p in a context in Γ, suppose a replacement $\rho(d)$ using q preserves the required interface, remains feasible, and satisfies $J(\rho(d)) \leq J(d)$. Removing the p designs in Γ preserves the minimum objective over that domain.*

Proof. The restricted set is a subset of the original set, so its optimum cannot improve. The replacement maps each removed design to a retained design with no worse objective, so its optimum cannot worsen. □

The useful engineering consequence is the scope of the rule. A dominance fact learned for one register budget or cache organization is stored with those conditions. A hardware change outside that context reopens the alternative. When the replacement conditions are only empirically suggested, exclusion is a heuristic search restriction and the branch remains available for reconsideration.

3.4 The software–hardware boundary as a variable

A boundary choice specifies which functions are fixed in hardware and which remain programmable. For a mechanism j, let $b_{j\mu}$ select one implementation mode μ: a software sequence, a programmable instruction mechanism, or a specialized controller. The modes share an external functional interface but have different hardware requirements and lowering rules. We impose

$$b_{j\mu} \in \{0, 1\}, \qquad \sum_{\mu \in M_j} b_{j\mu} = 1. \tag{5}$$

The choice may alter control area, instruction count, legal schedules, and supported tensor layouts. A capability is counted once in hardware and can serve many program regions. Each use pays its execution and contention costs.

This representation makes an otherwise fixed interface negotiable within an explicit library. It also distinguishes architectural sizing from boundary design: increasing queue depth changes capacity; introducing a completion-event interface changes what software can express. Both can be optimized, but they are different design decisions and require different implementation artifacts.

4 Constructing Coupled MIP Formulations

4.1 From computation graphs to implementation families

Figure 1 shows the proposed organization. A representation hierarchy connects workload semantics to algorithmic organizations, tensor programs, instruction interfaces, and hardware structure. The hierarchy supports local detail: a region whose performance is well characterized can remain a compact implementation family while a contentious region exposes its transfers, buffers, and individual resource demands.

A *family* is a parameterized implementation recipe. It specifies the computation it covers, external inputs and outputs, internal index and reduction structure, required capabilities, resource demand, and software and hardware lowering hooks. Parameters can include tile shape, buffering depth, layout, execution order, and instruction mechanism. Families can come from an established algorithmic library, compiler transformations, semantics-driven synthesis, or agent proposals admitted by the functional acceptance interface.

The agent's task includes identifying when the current families omit a relevant optimization paradigm. A model with only independent matrix multiplications cannot discover cross-operator streaming by

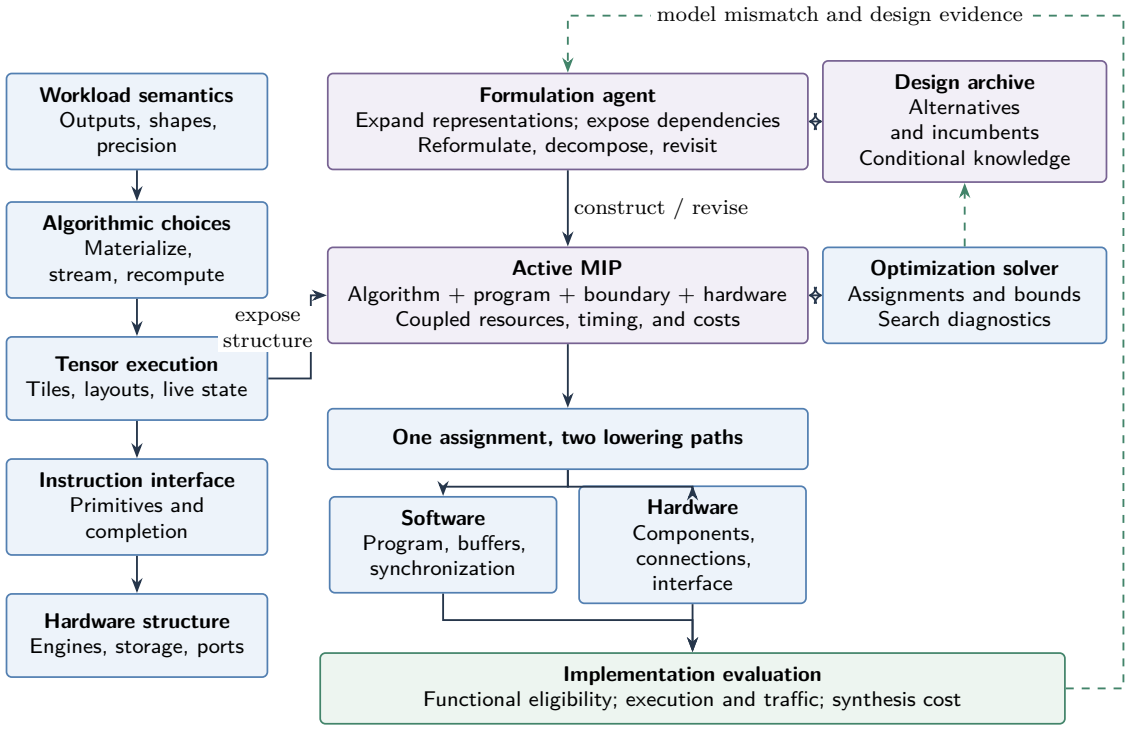


Figure 1: The proposed co-design loop. Representation levels on the left expose selected implementation choices; they do not prescribe an irreversible lowering order. Agents expand design capability and concentrate search. The same assignment determines software and hardware, and implementation evidence returns to the formulation policy.

adjusting their durations. A family must first expose the state and ordering that realize streaming. Hardware generation follows the same principle: choosing a specialized epilogue requires a component description and a corresponding program interface. A name attached to a binary decision variable does not supply its implementation.

Representation refinement therefore has two outputs. It adds executable structures, and it adds the relationships needed to price and constrain their combinations. Moving from graph-level edges to tensor slices reveals which values can be consumed early, which must remain live, and which can be recomputed. Moving from aggregate bandwidth to ports and queues reveals whether the proposed overlap is achievable. These relationships are the inputs to MIP construction.

4.2 Worked example: exposing streaming attention

Consider one attention head with query length N_q, key length N_k, and feature dimension d:

$$O = \mathrm{softmax}\Big(QK^\top/\sqrt{d} + M\Big)\,V, \tag{6}$$

where M is a mask and every output row has at least one unmasked key. A graph-level implementation may materialize both scores and probabilities. A streaming family partitions queries into row blocks I and keys/values into blocks J. Its state contains a running row maximum m, normalization denominator ℓ, and unnormalized output accumulator Z. For a score block $S = Q_I K_J^\top/\sqrt{d} + M_{IJ}$, the

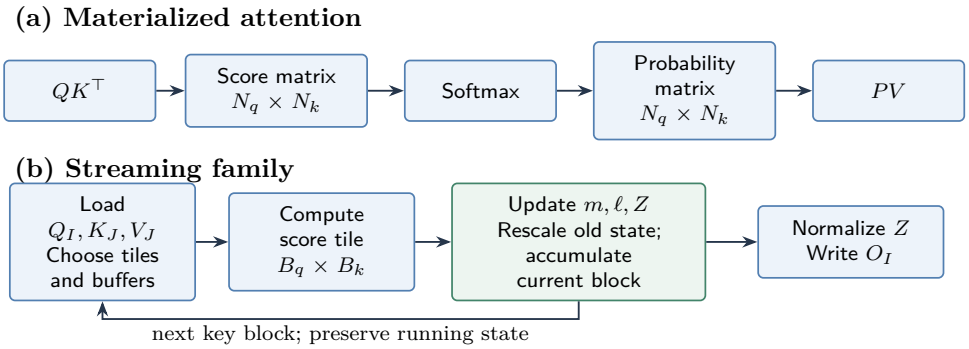


Figure 2: Exposing an algorithmic alternative requires new state and dependencies. The streaming recurrence avoids full intermediate matrices, while tile sizes, input reuse, buffering, and hardware support remain joint decisions.

update is

$$\begin{aligned} m' &= \max(m, \operatorname{rowmax}(S)), & \alpha &= \exp(m - m'), \\ P &= \exp(S - m'), & \ell' &= \alpha \odot \ell + \operatorname{rowsum}(P), \\ Z' &= \operatorname{diag}(\alpha) Z + P V_J. \end{aligned} \tag{7}$$

After the final key block, $O_I = \operatorname{diag}(\ell)^{-1} Z$. For each row, its first block containing an unmasked key initializes the state; blocks with no valid contribution leave that row's state unchanged. This is the established online-attention organization underlying IO-aware attention [16]; it is an example of a family to introduce, not a new attention identity.

The representation now exposes dependencies hidden inside the softmax node. Updating Z requires both the previous accumulator and the new rescaling factor. A score tile can be released after its contribution to normalization and output is complete. The output accumulator remains live across key blocks. Loading the next K, V block can overlap computation only when storage, transfer slots, and completion semantics support it. Work partitioning can further trade occupancy against shared-memory traffic, as illustrated by FlashAttention-2 [17].

For a tile with B_q query rows and B_k key rows, a conservative local working-set estimate is

$$B_{\text{local}} = s_Q B_q d + \nu(s_K + s_V) B_k d + s_S B_q B_k + s_Z B_q d + 2 s_r B_q, \tag{8}$$

where s_* are byte widths, ν is the number of buffered K, V blocks, and s_r is the running-statistic width. This expression assumes one score/probability buffer reused in place and excludes backend-specific scratch, alignment, and register spills, which the implementation family adds. Alternative reuse schedules produce different signatures. Byte counts should be distributed across the actual memory levels rather than charged to a single capacity if the lowering separates registers and shared memory.

For a fixed-query-block loop order, the K, V stream is traversed once per query block unless another level captures reuse. Hence reducing intermediate materialization does not imply that all input traffic is read once. Query grouping, cache behavior, and buffer capacity remain optimization decisions. At fixed tile and hardware modes, the storage and traffic expressions become coefficients; their joint selection remains in the MIP.

This example defines a progression from a coarse operator graph to an algorithmic family and then a resource-constrained execution structure. It also gives a concrete expressiveness test: the representation should generate both materialized and streaming variants and allow the optimizer to select between them under different resource budgets.

4.3 A finite linear encoding of coupled choices

We give a compact encoding for a bounded design domain. Let u index required semantic units, r candidate regions, $c \in C_r$ their implementations, i hardware slots, and $p \in P_i$ configurations for each slot. A slot may denote an engine, a memory component, or a parameterized subsystem. Local nonlinear expressions are evaluated for finite parameter combinations or represented with explicit linearization. They are not inserted into the MILP as unresolved products of design variables.

Coverage and selection. Let $q_{rc} \in \{0, 1\}$ select an implementation. For candidate regions covering a common semantic decomposition,

$$\sum_{r,c:\, u \in \mathrm{cover}(r,c)} q_{rc} = 1, \qquad \sum_{c \in C_r} q_{rc} \le 1. \tag{9}$$

These constraints select a nonoverlapping cover. Algebraically different organizations need a preceding family selection: each selected family activates its own internal decomposition. A materialized softmax graph and an online-state recurrence must not be forced to share artificial one-to-one internal nodes. Their common interface is the attention result, while their internal constraints are conditional on the selected family.

Hardware supply and implementation compatibility. Let $y_{ip} \in \{0, 1\}$ select at most one configuration per slot, with $\sum_{p \in P_i} y_{ip} \le 1$; an explicit absent mode can instead complete a one-hot selection. Resource supply and area can be written

$$\mathrm{Cap}_h(y) = \sum_{i,p} K_{iph} y_{ip}, \qquad A(y, b) = A_0 + \sum_{i,p} A_{ip} y_{ip} + \sum_{j,\mu} A^B_{j\mu} b_{j\mu}. \tag{10}$$

The component partition assigns each physical cost once; boundary-specific controller area is excluded from A_{ip} if it appears in A^B. Shared components are selected globally. For a required capability g, a selected implementation satisfies

$$q_{rc} \le \sum_{i,p:\, g \in \mathrm{cap}(i,p)} y_{ip}. \tag{11}$$

This establishes presence. Binding and cumulative demand constraints determine whether that capability has enough instances and service capacity. If placement or topology matters, bindings identify the instance that executes each operation, and transfers must connect its producer and consumer memories.

Timing and shared resources. For a time-indexed formulation, $x_{rct} = 1$ starts implementation (r, c) at tick t, with $\sum_t x_{rct} = q_{rc}$. Its duration d_{rc} and resource profile $U_{rch}(\delta)$ are fixed for that implementation mode. Then

$$\sum_{r,c,t} U_{rch}(\tau - t) x_{rct} \le \mathrm{Cap}_h(y) \quad \text{for every } h, \tau. \tag{12}$$

The profile can include overlapping compute, transfer, and epilogue stages. A hardware choice that changes latency must select a corresponding performance mode; it cannot silently change a coefficient after optimization. When contention changes operation order, the formulation introduces event ordering or a scheduling subproblem instead of assuming a fixed isolated duration captures that interaction.

Let $s_r = \sum_{c,t} t x_{rct}$ and $f_r = \sum_{c,t} (t + d_{rc}) x_{rct}$. If selected producer r and consumer r' exchange a value, their activation variable $g_{rr'}$ enables

$$s_{r'} \geq f_r + \delta_{rr'} - H(1 - g_{rr'}), \tag{13}$$

where $\delta_{rr'}$ is an explicitly charged transfer or visibility delay and H is a valid bound for deactivation. The activation is linked to both selected endpoints and the chosen data path. Fusion can remove an external transfer while retaining its internal dependency. Streaming permits tile-level consumers to start before the whole region completes, using tile events instead of this coarse edge.

Storage, ports, and overlap. A value-residency variable $\ell_{v\tau}$ is linked to the production and last consumption of value v. For fixed-size value objects,

$$\sum_{v} B_v \ell_{v\tau} + \sum_{r,c,t} S_{rc} (\tau - t) x_{rct} \leq \mathrm{Cap}_{\mathrm{mem}}(y), \tag{14}$$

where S_{rc} accounts for internal scratch not already counted as external values. The compiler determines which operands occupy registers, shared memory, or external storage, and analogous inequalities apply separately to each level. Port demand and outstanding transfers consume separate resources. Two buffers create room for overlap; they do not establish that the ports or queues can sustain it.

Objectives and multiple scenarios. For each scenario, L_w bounds the completion of its required outputs, including final visibility. With the objective in Equation (2), a linear epigraph uses $T \geq L_w / L_w^{\mathrm{ref}}$ and minimizes T. Each scenario has its own program variables and shares y, b. Area and configuration limits constrain the common machine. Dynamic energy requires per-action or activity-aware coefficients; variable power multiplied by variable runtime requires another linearization or external evaluation. Clock frequency must likewise be fixed or tied to an evaluated hardware mode when comparing time rather than cycles.

4.4 Encoding movable boundaries

Figure 3 illustrates a transfer mechanism. Software-controlled blocking loads, a programmable asynchronous interface, and a specialized streaming controller can implement the same data-flow requirement. They expose different choices to the program and consume different hardware resources.

An asynchronous implementation family requires the corresponding boundary mode, a compatible transfer engine, enough in-flight slots, and a buffer lifetime spanning completion. A specialized controller may reduce repeated instruction issue but accept only particular traversal patterns. The model links these choices with implications such as $q_{rc} \leq \sum_{\mu \in M_{rc}} b_{j\mu}$ and resource constraints for every active transfer. Choosing a mode also selects its lowering recipe. For instance, a programmable mode emits completion events and waits; a controller mode emits configuration and invocation commands.

The same construction applies to reductions, layout conversion, or fused epilogues. Hardware specialization pays a shared area cost, whereas software retains freedom at a recurring instruction and scheduling cost. The optimization compares these costs across all required scenarios. Appendix B gives a complete finite encoding of this choice, including configuration-dependent execution costs.

4.5 Decoding one assignment into paired implementations

The decoder creates a software execution plan and a hardware configuration from the same selected objects. The execution plan specifies loop structure, addresses, buffer ownership, transfers, resource bindings, and synchronization. The hardware description specifies component instances, capacities,

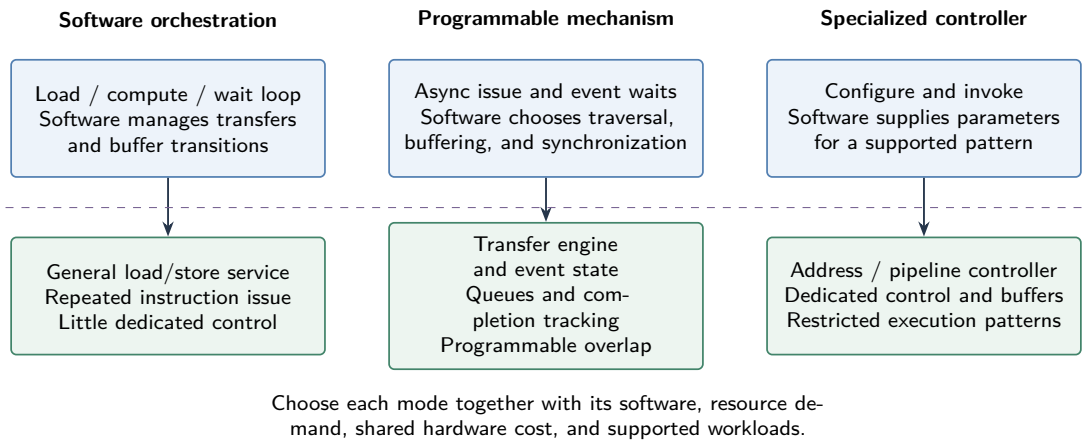


Figure 3: Three possible locations of transfer-control functionality. Each mode has a different software-visible contract and hardware realization. The optimization prices the remaining programmability through the workloads and capabilities that the design must support.

connections, and enabled interface mechanisms. A common identifier binds each emitted instruction capability to the implementing component.

For a parameterized existing architecture, the hardware output is a configuration of that generator. For a new boundary mechanism, the implementation library must also provide its controller or data path. The software backend cannot assume a mechanism that the hardware generator does not instantiate. Compilation then provides actual register and scratch requirements, which may differ from estimates used in the first model. Execution and synthesis return timing and implementation costs for the same paired design. These results close the loop by updating the family coefficients or exposing a missing constraint.

5 Agent-Guided Reformulation and Search

5.1 The agent operates on design decisions and model structure

The proposed agent receives the current representation, a typed mapping from variables to design objects, solver diagnostics, and implementation evidence. It produces a model action with an explicit scope. The action identifies which designs are added or restricted, which assumptions change, how an existing incumbent is reconstructed, and which evaluation can resolve the next uncertainty. This preserves the meaning of decisions when an encoding introduces many auxiliary variables.

The action space contains three groups. *Construction actions* add algorithmic families, expose tensor dependencies, or introduce hardware capabilities. *Reformulation actions* strengthen bounds, remove redundant variables, break valid symmetries, or choose a decomposition. *Search actions* release a coupled neighborhood, revisit a branch, select a warm start, or allocate an implementation evaluation. Cost-model revisions accompany these actions when the execution evidence changes the ranking assumptions.

This division matters operationally. A weak lower bound calls for a stronger mathematical representation. A fast solver repeatedly selecting slow implementations calls for a better performance model or a missing design family. A capacity failure after compilation calls for a resource constraint or another lowering. These observations should not all trigger a larger random search.

5.2 Structural reduction and coupled neighborhoods

Many variables in a detailed MIP are consequences of a smaller set of structural decisions. A selected tile, buffer organization, interface mode, and hardware allocation determine much of the remaining transfer and schedule structure. The agent can search this semantic core while a completion problem assigns times, memory slots, or detailed bindings.

A useful neighborhood releases decisions connected by a mechanism. For a register-limited matrix multiplication, these can include tile shape, registers per multiprocessor, resident workgroups, tensor shape, and the area allocated to other storage. Releasing only tile size may find no improvement because the enabling hardware change remains fixed. Releasing only register capacity may appear wasteful because the software does not exploit it. The neighborhood should preserve their coupling while fixing unrelated choices.

The formulation can exploit additional structure. Identical engine instances admit symmetry-breaking orderings if the workload and connectivity treat them interchangeably. Factorized loop mappings can avoid enumerating full schedules. Valid working-set inequalities can eliminate impossible combinations before introducing time-indexed variables. Derived demand variables can replace repeated expressions, and bounded binary–continuous products can use standard linearization. Each transformation is justified with respect to its declared domain; an instance-specific observation is not promoted to a universal rule.

5.3 Decomposition, completion, and bounds

A natural master problem chooses hardware, interface modes, and algorithmic families. Scenario subproblems choose detailed programs and schedules on that shared machine. This decomposition is useful when the master captures the strong global interactions and each scenario can otherwise be completed independently. Simultaneous multi-scenario execution requires a shared contention subproblem rather than independent completion.

A failed completion returns information about the combination that failed. If an exact subproblem proves a particular structural assignment infeasible, a no-good constraint can exclude that assignment. A stronger conflict cut can exclude a larger set only when its derivation supports that scope. A compiler error or a timed-out scheduling attempt does not establish infeasibility of every possible implementation with those high-level choices.

Primal and dual progress have different roles. A completed design gives a feasible objective for its evaluator. A solver lower bound applies to its formulation. For a branch B, a valid bound $\underline{J}_B \leq \inf_{d \in B} J(d)$ permits pruning when $\underline{J}_B \geq U$, where U is an accepted incumbent under the same objective and evaluator. An analytic model can support that statement if it is a certified relaxation of the target evaluation. A fitted prediction without a one-sided guarantee supports prioritization, not this bound-based exclusion.

Equivalent reformulations can retain bound meaning through a known mapping. Expanding the candidate family can invalidate a lower bound for the previously restricted problem; changing latency coefficients changes the objective being bounded. The archive therefore indexes bounds by model identity and scope. This prevents an inexpensive local optimum from being treated as a conclusion about the full design domain.

5.4 Retaining and revisiting alternatives

The candidate archive stores accepted implementations, unresolved family branches, and contextual knowledge. Structural signatures describe algorithmic organization, reuse direction, storage placement, boundary mode, and hardware allocation. Retention can then preserve distinct possibilities rather than many parameter variations of the same design.

An accepted design remains an incumbent when the representation expands, provided its workload and implementation conditions remain valid. A rejected design has a reason: illegal semantics, a resource violation, a provably dominated context, or an unfavorable evaluation. Only the last category is a performance judgment, and its scope includes the hardware and workload on which it was made. The packed-attention reversal in Section 6.3 illustrates why this distinction matters.

The archive also supports conditional knowledge reuse. A statement such as "this tile reuses a weight panel until register pressure reduces occupancy" contains a mechanism and applicability conditions. It can guide another scenario's initial formulation while leaving the threshold to be recomputed. A statement such as "larger tiles are always better" discards the conditions needed for transfer.

5.5 Feedback-driven action selection

The agent diagnoses three types of disagreement. *Feasibility disagreement* occurs when a modeled design cannot be lowered or violates a realized resource limit. *Ranking disagreement* occurs when implementation evaluation changes the ordering of candidates. *Search disagreement* occurs when a promising represented family remains difficult to explore despite an adequate evaluator. The responses are respectively to repair the interface or constraint, revise the relevant cost mechanism, or restructure the optimization.

Evaluation effort is directed toward decisions that can change the selected design. For example, two nearly tied memory organizations may justify detailed simulation of their contention, while a capacity-infeasible candidate can be removed analytically. When error bars are empirically estimated, they guide this allocation without becoming mathematical bounds. Expensive evaluations should include both promising designs and discriminating alternatives that test the agent's explanation of the bottleneck.

The procedure below specifies the proposed control loop. Its policy can begin with explicit rules and language-model proposals; a learned action policy is an optional implementation of the same interface.

Progressive co-design procedure

Input: workload set, component and transformation library, budgets, initial representation, and exploration budget.

1. Build an initial formulation and preserve a feasible reference design.

2. Diagnose missing families, solver difficulty, and implementation-model mismatch from the current record.

3. Propose a scoped construction, reformulation, or search action; retain its assumptions and reconstruction map.

4. Solve the active MIP or master/completion subproblems. Keep feasible assignments, bounds, and termination status.

5. Decode selected assignments into paired software and hardware. Check functional and resource eligibility, then evaluate execution and implementation cost.

6. Update coefficients, mechanism constraints, and contextual knowledge. Retain competitive structural alternatives and reopen affected branches.

7. Repeat while the exploration budget permits; return the best eligible evaluated design and its model-specific optimization information.

An evaluation of the agent should count the entire loop: formulation construction, language-model inference, solver calls, failed compilations, simulation, and synthesis. Solver time alone can improve while total exploration becomes more expensive. Conversely, an expensive model revision can be worthwhile when it is reused across many workload shapes. These are distinct amortization questions.

6 Preliminary Results

6.1 Workloads and experimental setting

The experiments examine whether joint design improves executable workloads, which software–hardware interactions account for the gains, and how revising the formulation helps search. Two platforms provide complementary evidence. A programmable architecture benchmark exposes compute arrays, register and shared storage, DMA concurrency, data layout, and attention organization. Experiments on the open-source Ventus GPU architecture compile OpenCL kernels to instruction streams and explore parameterized compute, cache, register, and memory-service resources. Each study compares candidates against its own fixed baseline under common resource limits.

The programmable benchmark uses five complete Transformer scenarios: two prefill workloads and three decode workloads with different batch sizes and context lengths. All five programs share one hardware configuration under a 100-unit modeled area budget and the benchmark's power limits. Private-seed checks validate numerical outputs and resource legality. The Ventus experiments use Qwen2.5-0.5B with 24 layers and a $1.11\,\mathrm{mm}^2$ modeled area budget. P128 and P512 denote prefill lengths; D128 and D512 average 16 decode steps starting at those context lengths. Selected designs complete instruction-level timing simulation of every layer and decode step, including packing, transfers, and the language-model head.

For Ventus scenario w, $r_w = L_w/L_w^{\mathrm{ref}}$ measures normalized cycles. Each task minimizes the worst ratio over its prefill, decode, or combined scenario set. The programmable benchmark instead reports a weighted geometric-mean speedup. Results are simulated cycles and modeled resources. Ventus executes control and address paths for timing, with separate local numerical checks; the programmable benchmark checks full workload outputs. The following experiments establish design mechanisms and executable candidates for the proposed agent loop. Appendix C records the normalization and evidence definitions.

6.2 Substantial gains from joint architecture and program design

The programmable benchmark reaches a weighted geometric-mean speedup of 7.02× over its supplied baseline. Individual speedups are 8.37× and 13.00× for the two prefill cases, and 4.02×, 5.22×, and 5.01× for decode (Figure 4a). The same selected hardware serves all five scenarios, so the aggregate includes the tradeoff between long attention, weight reuse, and decode concurrency. Compute-array shape, register capacity, data movement, and program organization are chosen together within the benchmark constraints.

On Ventus, a short exploratory campaign spanning three design iterations achieves 2.93× prefill and 3.58× decode speedups. The prefill and combined objectives select a design of estimated area $1.0812\,\mathrm{mm}^2$, with P128/P512 ratios of 0.3417/0.3308. The decode-oriented design occupies $1.0848\,\mathrm{mm}^2$ and reaches D128/D512 ratios of 0.2791/0.2578. The headline task ratios are determined by the 128-token scenarios and confirmed by complete instruction runs.

Controlled comparisons in the second iteration isolate each side's contribution. With the common slice evaluator, software-only optimization reaches prefill/decode objectives of 0.8581/0.9015, while

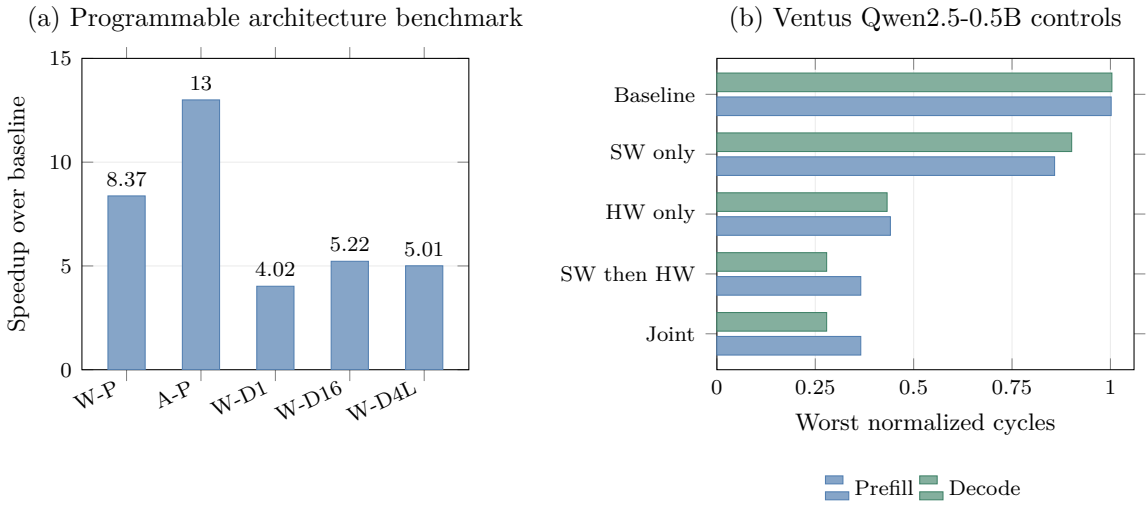


Figure 4: Preliminary co-design results. (a) One hardware configuration serves five complete Transformer scenarios and attains a 7.02× weighted geometric-mean speedup. Higher is better. (b) Second-iteration Ventus controls use the same slice evaluator and area budget; varying both software and hardware improves over changing either alone. Lower is better. Complete runs confirm the selected task results; the third iteration further improves prefill to 2.93×.

hardware-only optimization reaches 0.4409/0.4325. Allowing both to change reaches 0.3656/0.2789 (Figure 4b). Software-first selection ties joint selection in this evaluated menu. The comparison establishes the value of changing both sides; the search policy's additional value depends on the interactions and alternatives represented in its domain.

The software gains themselves depend on hardware. Across the reported comparisons, software changes reduce cycles by approximately 10–14% on the baseline machine. On the fixed optimized machine, paired P512 and D512 comparisons reduce cycles by approximately 18% and 41%. Column ordering, packing, and decode specialization therefore have greater value when the machine can support their reuse and memory-service demands.

6.3 Coupled choices require an evolving formulation

The performance gains expose concrete dependencies between program structure and resource allocation. Column-order GEMM places blocks that consume a common weight strip close together. Cache capacity then becomes useful where row-order sweeps had shown little benefit. Hardware search reallocates area toward outstanding memory requests and load/store queues. A decode-specific kernel loads only the nonzero fragment register for a single-row input, releasing load/store capacity without changing the matrix computation.

Larger tiles reveal a stronger interaction. Traffic analysis of a prefill projection finds 57 MB of DRAM reads for 8.7 MB of weights because row tiles reload the same data. Introducing a 64×64 GEMM kernel reduces traffic by roughly one third, but requires 173 vector registers per thread. On the earlier VGPR-512 configuration, normalized P128 cycles worsen from 0.362 to 0.438. A matching configuration with two multiprocessors, VGPR 1024, tensor shape $(4, 8, 4)$, and revised caches makes the tile useful within the area budget.

Hardware changes also reverse an earlier software decision. Packed QK is unfavorable on the baseline and the earlier three-multiprocessor design, yet improves P512 from approximately 0.377 to 0.334 on

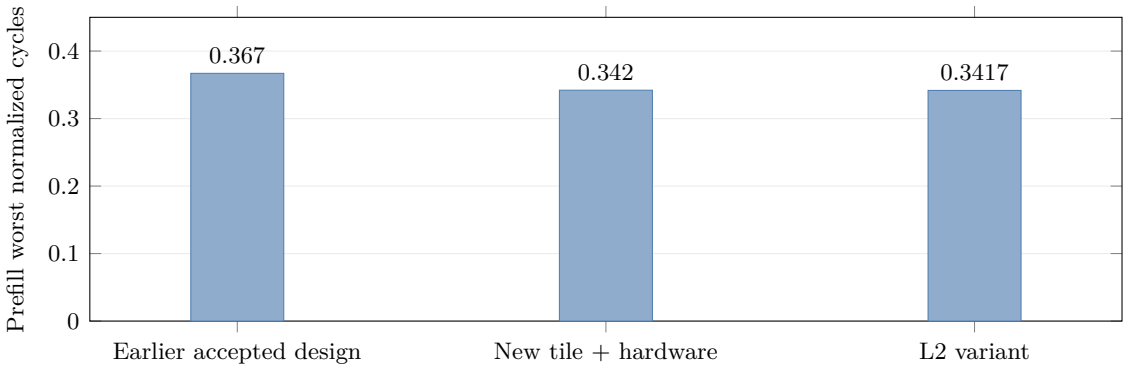


Figure 5: Accepted-design trajectory from complete instruction simulations. Introducing the larger-tile kernel and a matching register/tensor/cache configuration expands the available combination; an L2 variation then changes the selected hardware. The values are rounded as reported. This trajectory illustrates design evolution, rather than a controlled ablation of individual changes.

the new machine. A subsequent L2 adjustment produces the selected 0.3417 prefill objective. Figure 5 shows the accepted complete-simulation trajectory. Eliminating packed QK permanently after its first evaluation would remove a useful implementation from this later search space.

The formulation evolves alongside these implementation choices. An initial search stalls near a normalized prefill estimate of 0.433. Factorial experiments expose interactions among load/store concurrency, L2 miss-status entries, and write buffering; adding pairwise terms to the MIP helps reach approximately 0.362, a further 16% reduction. The larger tile then requires expanding the candidate family. The existing surrogate's penalty for poorly sampled choices keeps it from proposing the new combination, which targeted simulation subsequently finds and refines.

These observations connect representation construction to optimization. Program transformations change the resources worth buying, while hardware changes which transformations should remain available. An agent can respond by adding candidates, releasing a coupled neighborhood, and revising the interactions encoded in the formulation. Solving a fixed model more accurately alone cannot expose a kernel or hardware combination missing from that model.

6.4 Joint selection improves the storage–latency tradeoff

A separate Ventus experiment minimizes allocated storage for one Qwen KV-append step and all 14 attention heads of a layer. It combines ten area-feasible hardware candidates with software choices for K/V precision, quantization group size, grouped-query organization, and scratch reuse. The analysis covers 2,240 completed grid points. Storage accounting includes quantization scales, scratch buffers, alignment, and private stacks. Independent probe seeds check local attention-output error against the BF16 path.

At normalized latency limits around 0.18–0.20, joint selection saves approximately 16% or 17% of storage over software-only optimization under 2% or 5% local error thresholds, respectively. At tighter limits around 0.15–0.17, software-only selection becomes infeasible while joint designs remain feasible. Faster hardware preserves the option to use a more compact format under the latency constraint. Hardware-first selection matches the joint solution in this grid, giving a useful strong baseline for future search comparisons.

The result extends the co-design objective beyond speed. Hardware configuration, representation precision, and temporary-storage organization jointly determine which designs satisfy a workload contract.

The local error thresholds apply to this attention subproblem; whole-model quality is a separate deployment constraint.

6.5 Execution feedback identifies missing model structure

An independently constructed Ventus search provides a complementary diagnostic. Nine MIP solves close the gap in their approximate models and predict a worst-scenario ratio near 0.3333. Complete instruction simulation instead yields objectives between 0.7913 and 0.8987. Hardware-only selection matches or outperforms joint selection in that candidate set. Transferring the selected Qwen configurations to GPT-2 and Pythia also reverses the gains, with additional time concentrated in the FFN down projection and substantially increased memory reads.

This discrepancy identifies a concrete role for feedback. Solver optimality describes the current formulation; execution tests whether its traffic, reuse, and concurrency assumptions rank actual candidates correctly. The successful iterative study shows how adding such interactions and revisiting implementations improves the design. A progressive agent should preserve stronger evaluated candidates while changing the formulation responsible for a bad prediction.

Together, the results demonstrate substantial preliminary gains, hardware-dependent software preferences, and a storage-feasibility benefit from joint selection. They also provide specific actions for the proposed agent: expand an implementation family, revise coupling terms, and reopen a previously unfavorable branch. Evaluating the complete agent policy will compare these actions under common candidate access and total exploration budgets, measuring both resulting design quality and the cost of obtaining it.

7 Design Implications and Next Steps

7.1 From established paradigms to new combinations

The first test of a design representation is whether it can express competitive existing implementations. Attention provides a useful progression: materialized scores, tiled fusion, online normalization, different work partitions, and hardware-supported data movement. The representation should expose the mechanisms that distinguish these families, allowing them to combine with hardware choices. Encoding the name of an existing kernel as one opaque option offers a useful baseline, but provides little opportunity to recombine its internal decisions.

A richer representation creates the possibility of improving on established designs; it does not establish that improvement by itself. A frontier comparison needs a current strong implementation for the same workload, precision, and target constraints. Software comparisons hold hardware fixed. System comparisons allow hardware changes under a common resource and implementation-cost model. Reporting both separates algorithmic gains from additional hardware capability. The Ventus studies establish useful combinations within their declared domains; a claim of performance beyond state-of-the-art implementations requires that additional comparison.

7.2 How much of the boundary should move?

A software–hardware boundary should be reopened when its fixed assumptions prevent a useful design or impose repeated overhead. Reopening every interface simultaneously makes both implementation and optimization harder. The proposed approach introduces explicit alternatives for a selected mechanism, retains their programmable degrees of freedom, and evaluates the consequences over the required

workload set.

The current cases vary hardware resources and software organization within an existing interface. The boundary formulation in Section 4.4 extends that domain. An implementation study of boundary movement should generate the competing controllers or instruction mechanisms, compile matching programs, and include area and timing costs under a common technology and clock methodology. This would test whether specialization's recurring performance benefit justifies its fixed cost and reduced flexibility.

7.3 Cost fidelity and optimization scale

Fine-grained timing, bank conflicts, and dynamic arbitration can overwhelm a global MILP. A compact master model with conditional resource signatures and selective detailed completion is one way to manage this scale. It introduces a modeling choice: which interactions can be summarized without changing important rankings? The model-fidelity case shows why this question needs execution evidence.

The quality of the implementation library also limits the domain. Compiler register allocation, unsupported layouts, and backend synchronization rules can change feasibility after an initial model is built. These effects should produce explicit family restrictions or refined estimates. Physical cost introduces additional interactions, including memory macros, wiring, frequency, and control logic. A logical-area estimate and a cycle count do not determine an implemented chip's energy or latency.

7.4 Evaluating the agent and reusable knowledge

An agent can improve the design space, the formulation, or the allocation of search effort. These effects require different controls. With a fixed final space, compare equivalent formulations and structured search against strong solver and heuristic baselines. With an evolving space, give policies access to the same transformation and component libraries, then measure which designs they expose and how much evaluation effort they spend. A retention ablation should revisit the conditional reversals seen in the case studies.

Knowledge transfer is also conditional. A mapping rule learned on one architecture can identify a promising mechanism elsewhere, but its performance threshold depends on the new resource ratios. The negative transfer case argues for storing those conditions and validating them on reserved workloads. Human intervention and model-inference costs belong in the accounting. Reusable formulation knowledge is valuable when its construction cost is amortized over enough related design problems.

8 Conclusion

AI infrastructure design joins algorithmic choices, programs, interfaces, and hardware into one coupled optimization problem. Its difficulty lies both in representing valuable implementations and in searching the resulting combinations. Agentic Progressive Co-Design addresses these tasks together: introduce the mechanisms needed to express a useful design, encode their interactions in a MIP, exploit structure to make search tractable, and use paired implementation feedback to revise the next model.

The Ventus studies show that hardware changes can reverse software preferences, that changing both sides improves over optimizing each alone, and that a closed solver gap cannot repair an inaccurate performance model. These findings motivate a design process in which abstraction boundaries and optimization formulations remain revisable. The resulting research direction is a systematic route

from workload semantics to a matched algorithm, executable program, software–hardware interface, and hardware organization.

References

[1] Q. Huang, M. Kang, G. Dinh, T. Norell, A. Kalaiah, J. Demmel, J. Wawrzynek, and Y. S. Shao. *CoSA: Scheduling by Constrained Optimization for Spatial Accelerators*. ISCA, 2021. https://arxiv.org/abs/2105.01898.

[2] A. Kothari, S. Zhu, D. Kroening, and C. Sung. *Axon: A Synthesizing Superoptimizer for Tensor Programs*. arXiv:2606.26344, 2026. https://arxiv.org/abs/2606.26344.

[3] P. Jain et al. *Checkmate: Breaking the Memory Wall with Optimal Tensor Rematerialization*. MLSys, 2020. https://arxiv.org/abs/1910.02653.

[4] Y. Li, A. Gupta, and S. Malik. *Combined Scheduling, Memory Allocation and Tensor Replacement for Minimizing Off-Chip Data Accesses of DNN Accelerators*. arXiv:2311.18246, 2023. https://arxiv.org/abs/2311.18246.

[5] C. Lattner et al. *MLIR: A Compiler Infrastructure for the End of Moore's Law*. arXiv:2002.11054, 2020. https://arxiv.org/abs/2002.11054.

[6] S. Feng et al. *TensorIR: An Abstraction for Automatic Tensorized Program Optimization*. ASPLOS, 2023. https://arxiv.org/abs/2207.04296.

[7] L. Zheng et al. *Ansor: Generating High-Performance Tensor Programs for Deep Learning*. OSDI, 2020. https://www.usenix.org/conference/osdi20/presentation/zheng.

[8] M. Wu et al. *Mirage: A Multi-Level Superoptimizer for Tensor Programs*. OSDI, 2025. https://www.usenix.org/conference/osdi25/presentation/wu-mengdi.

[9] S.-C. Kao et al. *DiGamma: Domain-aware Genetic Algorithm for HW-Mapping Co-optimization for DNN Accelerators*. DATE, 2022. https://arxiv.org/abs/2201.11220.

[10] C. Hong, Q. Huang, G. Dinh, M. Subedar, and Y. S. Shao. *DOSA: Differentiable Model-Based One-Loop Search for DNN Accelerators*. MICRO, 2023. Author version: https://arxiv.org/abs/2509.10702.

[11] A. AhmadiTeshnizi et al. *OptiMUS-0.3: Using Large Language Models to Model and Solve Optimization Problems at Scale*. arXiv:2407.19633, version 4, 2026. https://arxiv.org/abs/2407.19633.

[12] H. Yuan, J. Peng, J. Bi, N. Zhang, S. Song, and Z. Cao. *FormuEvo: LLM-Guided Evolution for Discovering Solver-Efficient Mixed-Integer Programming Formulations*. arXiv:2608.23353, 2026. https://arxiv.org/abs/2608.23353.

[13] A. Parashar et al. *Timeloop: A Systematic Approach to DNN Accelerator Evaluation*. ISPASS, 2019. https://accelergy.mit.edu/timeloop.pdf.

[14] H. Genc et al. *Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration*. DAC, 2021. https://people.eecs.berkeley.edu/~ysshao/assets/papers/genc2021-dac.pdf.

[15] S. Karandikar et al. *FireSim: FPGA-Accelerated Cycle-Exact Scale-Out System Simulation in the Public Cloud*. ISCA, 2018. https://fires.im/.

[16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. NeurIPS, 2022. https://arxiv.org/abs/2205.14135.

[17] T. Dao. *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning*. ICLR, 2024. https://arxiv.org/abs/2307.08691.

A Notation and Linearization

Table 1 summarizes the objects used in the formulation. All finite ranges, time horizons, and coefficient versions are part of a model instance.

Symbol	Meaning
$\mathcal{W}, \mathcal{D}_W$	Required deployment scenarios and feasible bounded implementation domain
$R_k, \mathcal{F}_k, \mathcal{R}_k$	Active representation, feasible assignments, and decoded realizable designs
a, p, m, σ	Algorithmic organization, program, mapping, and execution organization
h, b	Shared hardware structure and software-visible boundary choices
q_{rc}, x_{rct}	Implementation selection and time-indexed start decisions
$y_{ip}, b_{j\mu}$	Hardware-slot configuration and boundary-mode selection
U_{rch}, S_{rc}	Local resource-demand and internal-scratch profiles
$\ell_{v\tau}, B_v$	Value residency and fixed byte footprint
$L_w, \widehat{J}, J$	Scenario latency, modeled objective, and target evaluator objective

Table 1: Main notation. A symbol denoting a program p is distinct from the configuration index $p \in P_i$ in hardware-slot sums.

Logical conjunction. For binary selections q, y, an auxiliary $z = q \wedge y$ is encoded by

$$z \le q, \quad z \le y, \quad z \ge q + y - 1, \quad z \ge 0. \tag{15}$$

This is useful for configuration-dependent duration or energy coefficients. It enumerates local interactions without requiring an opaque candidate for every whole-system combination.

Binary–continuous products. For $z = qv$ with $q \in \{0, 1\}$ and known bounds $L \le v \le U$, the exact linearization is

$$Lq \le z \le Uq, \qquad v - U(1 - q) \le z \le v - L(1 - q). \tag{16}$$

Tight bounds matter for relaxation strength. A generic large constant can make a valid model unnecessarily difficult to solve.

Residency. Residency variables cannot be free capacity switches. For a selected implementation, they are constrained to remain active from production or preallocation until every required consumer completes. One finite encoding enumerates allowed start/last-use pairs and derives the active interval; another uses production and release events with cumulative conservation. Aliasing and buffer reuse require nonoverlapping live ranges or explicit ordering. Internal scratch and exported values must not be counted twice.

Configuration-dependent timing. A single duration coefficient is valid only for the hardware conditions under which it was defined. A finite (implementation, hardware mode) table can use conjunction variables. A factorized model instead represents the relevant service capacities explicitly and derives event times. The former is compact when interactions are local; the latter is needed when relative order and interference change with the configuration.

Conditional families. Suppose an attention region has family selectors $g_{\mathrm{mat}}, g_{\mathrm{stream}}$ summing to one. Each internal operation u belonging to family a satisfies $\sum_{r,c:\, u\in \mathrm{cover}_a(r,c)} q_{rc} = g_a$. The selected family exports the same region-level inputs and output. Transfers and lifetimes are internal to that family except for shared external values. This avoids treating an online recurrence as a cover of intermediate tensors that it intentionally does not materialize.

B A Complete Finite Boundary-Choice Model

Consider a region that must transfer and consume N tiles in each workload scenario. Let $\mathcal{H}$ be a finite set of machine configurations and $\mathcal{M} = \{\mathrm{sw}, \mathrm{async}, \mathrm{ctrl}\}$ the permitted transfer-control modes. Each hardware/mode pair has a generator and compiler interface. Let $\mathcal{C}_w$ contain software variants with fixed tile sizes, traversal order, and buffering depth for scenario w.

The coefficients are: shared area A_h, incremental mode area $A^B_{h\mu}$, compatibility $K_{wh\mu c} \in \{0,1\}$, and evaluated or modeled latency $t_{wh\mu c}$. The mode-area coefficient includes only components not already counted in A_h. Compatibility includes numerical eligibility, supported traversal, live storage, resource demand, and interface support. Thus a specialized controller may support fewer variants than the programmable mode.

Choose binary hardware y_h, mode b_μ, and software q_{wc} variables with

$$\sum_h y_h = 1, \qquad \sum_\mu b_\mu = 1, \qquad \sum_{c\in\mathcal{C}_w} q_{wc} = 1 \quad \forall w. \tag{17}$$

Let $v_{h\mu} = y_h \wedge b_\mu$ and $z_{wh\mu c} = v_{h\mu} \wedge q_{wc}$ using the conjunction encoding above. The complete selection model is

$$\begin{aligned}
\min \quad & T, \\
\text{subject to} \quad & z_{wh\mu c} \le K_{wh\mu c} && \forall w,h,\mu,c, \\
& \sum_h A_h y_h + \sum_{h,\mu} A^B_{h\mu} v_{h\mu} \le A_{\max}, && \\
& L_w = \sum_{h,\mu,c} t_{wh\mu c} z_{wh\mu c} && \forall w, \\
& T \ge L_w / L^{\mathrm{ref}}_w && \forall w.
\end{aligned} \tag{18}$$

Hardware and boundary mode are shared, while programs differ across scenarios. The area of an asynchronous engine is paid once even when several scenario programs use it. Its per-scenario effect is represented in t and K. Latency entries must include startup, queue occupancy, synchronization, and the attainable overlap for the selected variant.

This finite model is an exact optimizer of its supplied table. Its purpose is to make the boundary decision and accounting explicit. Scaling to a large family replaces parts of the table with factorized resource and event constraints, and generates additional variants as needed. Doing so changes the computational representation while retaining the shared implementation identities. A boundary experiment would populate these coefficients from generated software and hardware; the empirical studies in Section 6 retain their existing instruction interface.

C Empirical Definitions and Source Records

The accompanying artifact retains versioned reports, iteration notes, complete-run summaries, and the programmable benchmark's verified result. New evidence records supplement the earlier snapshots

without replacing them. Numerical tables reproduce the plotted values without rerunning accelerator search. The evidence directory records source locations and the scope of each evaluator.

Programmable benchmark. The five scenarios are W-P, A-P, W-D1, W-D16, and W-D4L. W uses two layers, width 1536, 12 heads, and FFN width 6144; A uses two layers, width 512, eight heads, and FFN width 2048. Prefill lengths are 256 and 2048. The decode cases use batch/context pairs $(1, 4096)$, $(16, 256)$, and $(4, 4096)$ for four steps. The shared hardware satisfies a 100-unit area limit and modeled power limits of 34 W over 100 cycles and 26 W over 10,000 cycles.

Frozen baseline cycles are 187689274, 374856358, 27068433, 42358913, and 70439292. The verified candidate cycles are 22415366, 28836408, 6732470, 8109251, and 14069101. With weights 1/4, 1/4, 1/6, 1/6, and 1/6, the aggregate is

$$\exp\left(\sum_w \omega_w \log \frac{L_w^{\mathrm{ref}}}{L_w}\right) = 7.017319.$$

The frozen result uses model `phase-two-compact-v2` and passes private-seed verification. The reported cycles describe the simulated target, independently of host evaluation time.

Ventus controls and complete runs. Figure 4b uses second-iteration slice estimates in the accompanying control table. The baseline estimates are 1.0019 and 1.0037; their small deviations from one reflect the slice evaluator. Software-first and joint selection tie at 0.3656/0.2789. Complete runs of the selected designs give 0.3669/0.2791 before the third iteration improves prefill to 0.3417. Decode slices warm up the prior step's tail to represent continuous cache state.

The headline speedups are $1/0.3417 = 2.9265$ and $1/0.2791 = 3.5829$. Every selected scenario executes all layers and decode steps. The frozen 128-token baseline is reproduced directly; the historical 512-token baseline is extrapolated. The 128-token cases determine both headline task objectives. Full instruction timing does not imply full-network numerical inference: local replay and address/workload checks provide separate functional evidence.

Tiling trajectory and formulation changes. Figure 5 preserves accepted complete-simulation prefill values 0.367, 0.342, and 0.3417. The last design has ratios 0.3417, 0.3308, 0.2953, and 0.2719 and estimated area $1.0812\,\mathrm{mm}^2$. The separate decode design has ratios 0.2791 and 0.2578 and estimated area $1.0848\,\mathrm{mm}^2$. Slice-based mechanism comparisons are identified separately from this complete-run trajectory. The three iteration records retain hardware reversions, software comparisons, the pairwise surrogate revision, and targeted exploration of the larger-tile family.

Storage and model fidelity. The KV study covers one layer's append and 14 attention heads. Its 2,240 completed grid points evaluate ten hardware candidates and 56 software variants across the study's settings. Local quantization-error filtering uses separate calibration, selection, and final-check seeds; its thresholds do not measure whole-model quality. The model-fidelity study retains a separate evaluator and fixed-hardware GPT-2/Pythia transfer tests. Its ratios are normalized to each workload's own baseline. The two studies are interpreted independently of the headline timing experiments.